

KI-drevet utvikling

Hvordan vi jobber med Agenter og skills i Kai

Bjørn Kristian Punsvik
Team Virkemiddel og KI
2026-08-19



Hvorfor dette?

Før

- Grovarbeid stjeler tid fra erfarne blikk
- Kunnskap låst i hoder — DevOps, database, backend
- Endring på tvers av alle lag gjøres manuelt, fil for fil

Nå

- Agenten tar gravarbeidet — parallelt — vi styrer
- Kunnskap kodet som skills → dukker opp i andres agent-sesjoner
- Nytt felt gjennom alle lag (frontend → API → domene → DB) i ett løp

Visjon

KI er en kollega i hele utviklingsløpet — ikke en autocomplete.

Arbeidsmåten vår er **kodet som skills**: delt, versjonert, gjenbrukbar.

Felles grunnlag

Begrepene — og hvor mennesket skal stå

Begreper — modellen og motoren

LLM — språkmodellen selv (Opus, Fable, Haiku). Kan bare én ting: lese tekst, gjette neste tekst. Ingen minne, ingen tilgang.

Context window — alt modellen ser i ett kall: prompt, filer, tool-svar. Fullt vindu = dårligere svar. Derfor delegerer vi.

Harness — programmet rundt modellen (Claude Code). Gir den tools, kjører loopen, holder styr på filer og permissions.

Tools — det harnesset lar modellen gjøre: lese fil, redigere, kjøre Bash, kalle API. Uten tools: bare prat.

Agent = LLM + harness + tools + oppgave, i loop til den er ferdig.

Begreper — laget vi bygger selv

Skill — markdown-fil med instruks: «slik gjør vi X her». Agenten plukker den selv når oppgaven passer. Ligger i repoet, versjonert.

Plugin — pakke med skills som deles på tvers av repoer (`superpowers`).

MCP — standard for å koble agenten til eksterne systemer (ADO, Microsoft Learn) uten å skrive egen integrasjon.

Subagent — egen agent med eget context window, får én avgrenset oppgave, leverer svaret tilbake. Hovedtråden holdes ren.

Worktree — isolert kopi av repoet på egen branch. Én per work item → flere agenter jobber samtidig uten å trække på hverandre.



Hvor skal innsatsen ligge? (DDD)

Core domain

Det som gjør Enova til Enova.
Regelverk, virkemidler,
saksbehandling.

Ingen kan kjøpe det. Feil her er
dyre.

Supporting

Nødvendig, men ikke særegent.
Rapportuttrekk, admin-flater,
importjobber.

Må finnes, må stemme — men
gir ikke konkurransefortrinn.

Generic

Løst for alle: auth, logging,
CRUD, CI/CD, setup, migrasjoner.
Hyllevare. Verdien er at det bare
virker.

Domain-Driven Design deler systemet etter **hvor verdien faktisk sitter** — ikke etter teknisk lag.

Derfor: menneske i core, agent i resten

Mennesket eier core domain

- Forstå regelverket og hva saksbehandler faktisk trenger
- Modellere domenet, sette navn og bounded contexts
- Arkitektur og beslutninger som er dyre å reversere (`adr-skill`)
- Agenten skriver gjerne koden — men vi eier *hva* som skal lages

Agent + skills tar supporting og generic

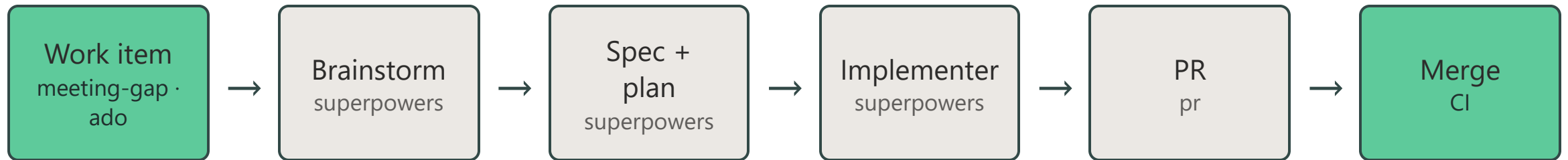
- Boilerplate, CRUD, tester, migrasjoner, setup
- Nytt felt gjennom alle lag i ett løp
- Drift og gravearbeid: pipeline-logger, KQL, SQL
- Skills koder *vår* variant av hyllevaren, så den blir lik hver gang

Fella: agenten er like rask i core domain — og derfor er det der du må bremse mest.

Loopen vår

Fra work item til merge

Hele løpet på ett bilde



Hvert steg styres av en **skill** — `superpowers` -pluginen driver kjernen, repo-skills gir domene- og driftskunnskap.

Skills — arbeidsmåten i koden

- **To lag:** `superpowers` -pluginen (arbeidsmåten, delt på tvers) + `repo-skills` (domene/drift for kai)
- Versjonert sammen med koden — review i PR som alt annet
- Agenten plukker riktig skill ut fra hva du ber om

22 repo-skills i dag — et utsnitt:

- Loop: `ado` · `pr` · `branch-hygiene`
- Azure/drift: `ado-pipeline-logs` · `appinsights-kql` · `entra-access` · `deploy-cu`
- Data: `databricks-sql` · `kai-postgres-connect`
- Domene: `kai-docs` · `regelmotor-regel` · `zensical-authoring`
- Arkitektur: `adr-skill` · `api-design`



Steg 1 — Work items i Azure DevOps

meeting-gap + ado · fra møte/observasjon → nedbrutt i ADO

- Behov kommer fra møtetranskripsjoner (meeting-gap) eller observerte bugs/features
- ado -skill bryter ned til Feature → User Story → Task, lenker parent-child
- Feature fanger *hvorfor* (verdi for saksbehandler), User Story fanger *hvordan*



Claude Code v2.1.232
Opus 5 (1M context) with high effort · Claude Team
~\enova\kai

/ado 21342 /brainstorming

```
Extracting WI 21342 from cache
L $ uv run python -c "
import json
d=json.load(open('.azure-devops/work-items.json',encoding='utf-8'))
def walk(items):
for it in items:
yield it
for k in ('children','user_stories','tasks','bugs'):
if isinstance(it.get(k),list): yield from walk(it[k])
root = d if isinstance(d,list) else d.get('featu...
```

* Wandering... (16s · ↓ 766 tokens)

» auto mode on (shift+tab to cycle) · esc to interrupt /rc

Steg 2 — Brainstorm & scope

`superpowers:brainstorming` · interaktiv scoping

- Agenten stiller spørsmål via **AskUserQuestion** — vi tar avgjørelsene
- Scope defineres sammen FØR noen skriver kode
- Ingen egne feature-dokumenter lenger — alt går gjennom superpowers-flyten

```
←!— Notat:  
[REGI] Avrund loopen. Vi er tilbake til start – klar for neste work  
item.  
→  
  
---
```

- Steg 2 fortsetter – de tre avgjørelsene jeg trenger fra deg før jeg foreslår design:

← ☒ Judgment ☐ Kontrakt ☐ Frosne rader ☒ Submit →

Skal judgment-reglene (LLM-baserte) med i denne US-en, eller holder det med de strukturerede?

1. Begge, best effort for judgment (anbefalt)
Strukturerst sti blir deterministisk (ekstraktoren eier journalposten). Judgment kobler LLM-ens dokumentnavn tilbake til fakta-recorden: entydig treff gir journalpost, tvetydig gir null og dagens oppforsele. Ingen prompt- eller skjemaendring, ingen ny LLM-feilmodus.
2. Kun strukturerede regler
Minst diff, men AC-en holder ikke for TilbudGyldighet, KontraktSignering, MvaSamsvar og AntallKjoretøy – som er en reell del av TN-flaten i dag.
3. Begge, deterministisk hele veien
Fakta-recorden far en dokumentnøkkel som judgment-output-skjemaet ma ekko. Deterministisk, men rorer prompter, output-skjema og golden-set for 4+ regler, og legger identitetsansvar pa modellen.
4. Type something.

5. Chat about this

Enter to select · Tab/Arrow keys to navigate · Esc to cancel

Steg 3 — Spec + plan

superpowers · spec + plan → venter på godkjenning

- Utforsker kodebasen, foreslår teknisk design + plan mot akseptansekriteriene
- **Approval gate**: ingenting implementeres før mennesket sier ja

Design-smell – les før neste lapp

Dette er tredje gang navnet lappes som join-nøkkel: 2026-07-29-dokument-tittel-i-regelvisning, 2026-07-30-opprinnelig-filnavn-dokumentnavn, og nå denne. Lappene var riktige hver for seg – problemet er at dokumentidentiteten aldri ble modellert, bare navnet forbedret.

Etter denne endringen er regelen: navn er visning, nøkkel er identitet. En ny join på navn i denne kjeden er en regresjon, ikke en snarvei.

Spec self-review kjørt: ingen TBD-er, ingen motstrid mellom seksjonene, scope holder til én plan. Ett punkt strammet inn underveis – «fallback for gamle rader» var tvetydig og kunne leses som midlertidig kode; nå eksplisitt permanent.

Les gjennom, så tar vi den videre til `superpowers:writing-plans`. Noe du vil endre først?

* Cogitated for 59s

> ser bra ut – lag planen

● Skill(superpowers:writing-plans)
 L Successfully loaded skill

* Working... (2m 2s · ↓ 5.3k tokens)
 L Tip: Use /btw to ask a quick side question without interrupting Claude's current work

> |

▶ auto mode on (shift+tab to cycle) · esc to interrupt

/rc

Steg 4 — Implementer

`superpowers:subagent-driven-development · plan → kode`

- Jobber mot akseptansekriteriene, reviewer i småsteg — ikke én diger diff
- **TDD-loop**: kjør til tester, linting og format er grønt — ellers på nytt
- Støtte: `api-design`, `adr-skill` når en beslutning er arkitektonisk

Task	Modell	Hvorfor
1, 2	Sonnet / low	Planen inneholder koden – transkribering + test
3	Sonnet / medium	Fem filer, samme endring, men gruppering må forstås
4	Sonnet / medium	To konstruksjonspunkter, ett er lett å overse
5	Opus / medium	Eneste task med reell dømmekraft – entydig vs. tvetydig
6, 7	Sonnet / medium	Kontraktsendring + integrasjonstest
8	Sonnet / low	En join-funksjon + Kiota-regen
Sluttreview	Opus / high	Hele branchen under ett

Task 1 og 2 er samme form – små, uavhengige feltutvidelser med koden ferdig i planen. De batches til én dispatch og reviewes som én diff, ikke to.

Klart til å dispatche. Skal jeg kjøre løpet?

* Cooked for 45s

> |

▶▶ auto mode on (shift+tab to cycle)

/rc

Steg 5 — Pull request

`pr` · oppretter ADO-PR fra branch → main

- Lenker work items automatisk, støtter draft
- Branch policies / build validation kjører før merge (`ado-pipelines`)

- Bruker `pr`-skillen. Beskrivelsen skrives til UTF-8-fil og sendes via REST – `az repos pr --description` mangler øøå.

PR-tittel
`fix(kai-infrastructure): link rule documentation to the journalpost the rule evaluated`

Beskrivelse

Problem

When a case has two journalposter with files sharing a filename, "etterprøv i originaldokument" in the rule analysis linked to an arbitrary one of them – wrong document, or a 404.

Root cause

Document identity in this system is (JournalpostVisningsId, Filnavn) – the pair DbRuleQueries already uses in DistinctBy. The identity is available all the way out in the extractor: FaktaSegment.SourceDocument is the blob path, and that path is built from the journalpost.

DocumentDisplayName() trimmed the path to its last segment. From there on, the display name did double duty as identity, and every consumer guessed its way back – server-side (docByNavn, first-wins), in frozen ReasonDetail, and in the frontend (document.name = Jump to bottom (ctrl+End) ↓

› |

▶▶ auto mode on (shift+tab to cycle)

/rc

Steg 6 — Merge & cleanup

Merge til main

- Squash-merge
- God test coverage + CI: grønn PR-pipeline → lite behov for menneskelig review
- Unntak: be om review fra den som kan området best

branch-hygiene

- Finn og rydd merged/stale branches trygt
- Forstår at squash-merge skjuler merged branches

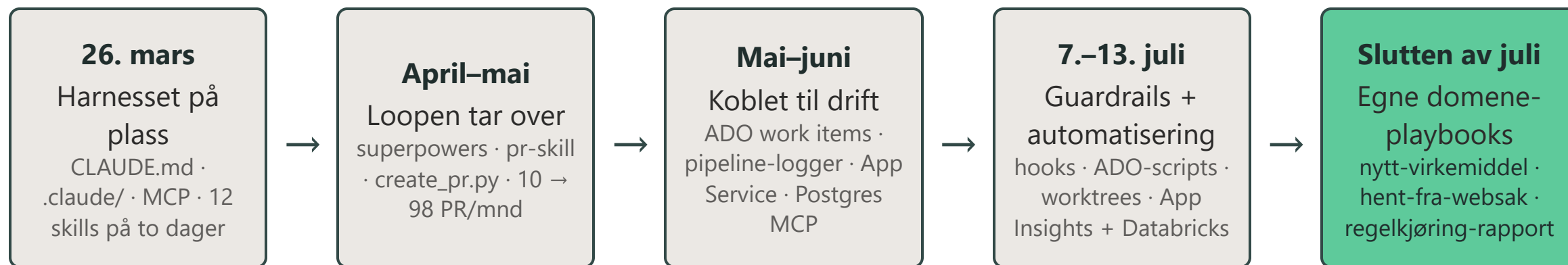
Loopen er ikke teori

Hvor ofte hvert steg faktisk kjørte (skill runs, 7 uker):

Steg	Skill	Runs
PR	<code>pr</code>	73
Brainstorm	<code>brainstorming</code>	67
Plan + implementer	<code>writing-plans</code> + <code>subagent-driven-development</code>	69
Work item	<code>ado</code>	56
Merge & cleanup	<code>finishing-a-development-branch</code>	15



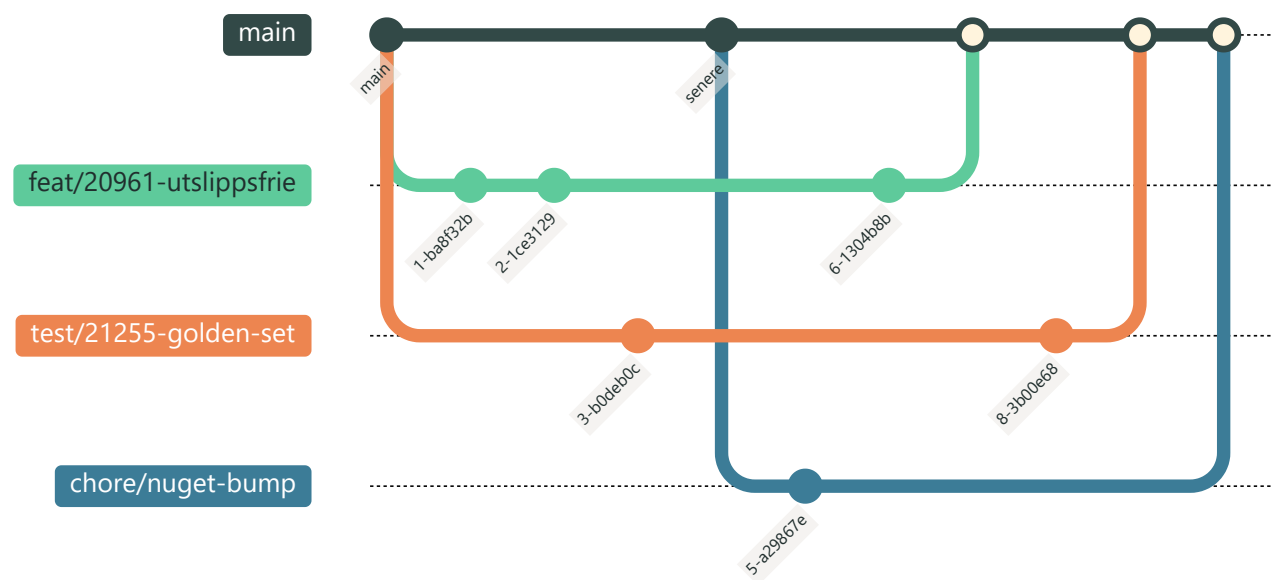
Utviklingen — mars til august 2026



Fra **generisk harness**, til **operasjonell tilgang** på reell prosjekttilstand, til **domenespesifikke playbooks** som kjører Enova-arbeidsflyter.

Parallellisering med git-worktrees

Én **worktree per work item** — isolerte kopier av repoet. Flere agenter jobber samtidig, uten å tråkke på hverandre eller bytte branch.



chore/nuget-bump startet **etter** de andre — nye spor er gratis · **53 av 127 sessions kjørte i worktree**

7 uker i tall

127

sessions

~1 680

prompts

~38M

tokens generert

53

i git-worktrees

633

subagent-kall

Opus 4.8

Fable 5 · Haiku

Hentet fra Claude Codes egne session-logger

«Men hvor fritt får den gå?»

Guardrails rundt agenten

Koden sier ifra

- Tester, kjørt i loop
- Linter + formatter
- **Arkitekturtester** — hvem importerer hva
- Typecheck / build

Harneset sier stopp

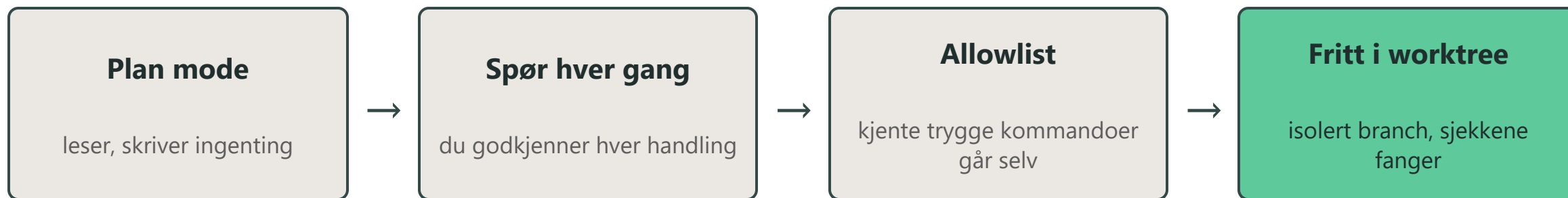
- Hooks blokkerer farlige kommandoer
- Allowlist — spør om resten
- Approval gate på spec + plan
- Ofte read-tilgang

Prosessen holder

- Worktree — isolert kopi
- PR + branch policies + build
- Squash-merge til main
- Alt i git — alt reversibelt

Vi stoler ikke på modellen. Vi stoler på **sjekkene den må gjennom** — de samme som gjelder oss.

Autonomi er en skala, ikke en bryter



Blast radius er én branch. Verste utfall: slett worktreen og start på nytt. Prod nås bare gjennom deploy-pipelen — samme vei som før.

Ansaret flytter seg ikke. Du eier PR-en, du eier koden, du eier feilen.

Guardrails bygger seg selv

Den positive loopen

- Testene, linter-reglene og arkitekturtestene som holder agenten i sjakk — er skrevet av agenten
- Akkurat det arbeidet som aldri ble prioritert før: «vi tar det senere»
- Bedre guardrails → tør gi mer autonomi → mer kapasitet → bedre guardrails

Dokumentasjon som lønner seg dobbelt

- Endelig tid til å skrive den — og den blir lest
- Mennesker tar riktigere valg, agenten tar riktigere valg
- Skills er dokumentasjon som **kjør**er: `kai-docs`, `api-design`, `adr-skill`
- ADR-er fanger *hvorfor* — agenten slutter å foreslå det vi allerede forkastet

Hva funker — og fallgruver

Funker

- Mye høyere test coverage — og guardrails rundt
- Frigjør fokus → mer gjort, og ting man ellers ikke ville gjort
- Kodehygiene + refactoring → tid spart, færre feil
- Sparer mest på store refactorings: coverage som sikkerhetsnett, fanger filer du glemte

Fallgruver

- Mennesket må styre HVA som lages
- Fokuser mennesket på core domain (DDD)
- «Kode er billig»-fella: det du lager setter presedens for agentene
- Eks.: in-memory-modus ble dobbeltarbeid mot DB da appen vokste → fjernet

Guardrails fanger **feil kode**. De fanger ikke **feil løsning** — det er fortsatt din jobb.

Kom i gang med det du har

1 · Installer og start

Copilot du kjenner lever i editoren. Dette er samme abonnement, men som agent i terminalen — den kan lese og endre filer selv.

```
winget install GitHub.Copilot
copilot # stå i mappa til repoet
```

2 · Be om noe helt vanlig

Skriv oppgaven i klartekst, som til en ny kollega: «*legg til feltet X i skjemaet, med test*». Agenten foreslår, du godkjenner.

3 · Skriv ned det du ellers forklarer muntlig

Agenten leser disse filene automatisk hver gang — det er teamets huskereglar, ikke kode:

- `AGENTS.md` i rota — kort om prosjektet: hva det er, hvordan man kjører testene, hva man ikke skal røre
- `.github/copilot-instructions.md` — samme idé, GitHub-spesifikk
- Egne regler for én mappe hvis frontend og backend har ulike vaner

Én fil er nok til å begynne. Det er din første skill.



Slik bruker vi KI

Agenten kjører hele loopen — work item, brainstorm, spec, kode, PR — og arbeidsmåten ligger i repoet som skills.

Guardrails fanger feil kode. Vi bruker tiden vår på core domain og på å avgjøre hva som faktisk skal lages.

Takk

Spørsmål?

Bjørn Kristian Punsvik
Team Virkemiddel og KI

